Machine learning: Open problem in churn prediction

In the telecommunications business, "churn" refers to the movement of customers from one service provider to another. Service providers want "low churn" -- they want their customers to stick with them.

Can a service provider predict who will discontinue service based on factors like whether they are a senior, how long they have been a customer, whether they have internet security software, and how much they pay each month? Obviously, a service provider would like to be able to predict if a customer has a high probability of leaving.

You will complete this notebook by using the methods we have covered in class to try to predict the 'Churn' variable, which is a binary variable.

The dataset has about 7,000 rows, with each row representing a customer. The variables in the dataset tell us things like how long the customer has been with the service provider, what kind of contract they have, and whether the customer is a "senior citizen".

Instructions:

- Use two machine learning algorithms to predict the value of the target variable 'Churn'.
- You will need to preprocess the data to prepare it for machine learning.
- You are expected to use the methods and best practice we've covered in class.
- The use of AI like ChatGPT for explicit solution is not allowed in this assignment, but you can use AI to help you understand task and give suggestions.
- You do not need to provide lots of text, but provide clear, brief text to explain what you are doing.
- You will need to add imports to the imports code cell below.
- Run your notebook from top to bottom before submitting. Cell execution numbers must be visible.

Refer to class materials to remember our recommended best practice for:

- data scaling
- test sets and cross validation
- assessing your prediction results
- hyperparameter tuning
- encoding categorical variables

For this notebook, you don't need to use or worry about:

- data visualization, except as you need for identifying variable types
- missing data

Tidiness of code and text is important, though. Please separate your notebook into sections using section headings. Use '### ' to indicate subheadings, as in this cell.

```
# Some commonly-used packages are imported below
import warnings
```

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
# import LogisticRegression
from sklearn.linear_model import LogisticRegression
# import KNeighborsClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
# this is in preparation for Pandas 3.0
pd.options.mode.copy_on_write = True
```

```
# plotting
sns.set_theme(style='whitegrid', context='notebook')
plt.rcParams['figure.figsize'] = 5,3
```

```
# suppress Seaborn future warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
```

Read the data

```
df = pd.read_csv("https://raw.githubusercontent.com/grbruns/cst383/refs/heads/maste
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   OnlineSecurity    7043 non-null   object
 1   DeviceProtection  7043 non-null   object
 2   PaperlessBilling  7043 non-null   object
 3   SeniorCitizen     7043 non-null   int64
 4   tenure            7043 non-null   int64
 5   MonthlyCharges    7043 non-null   float64
 6   Contract          7043 non-null   object
 7   Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(5)
memory usage: 440.3+ KB
```

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

Data preprocessing

```
# Convert target to Binary
df['Churn'] = df['Churn'].map({'No': 0, 'Yes': 1})
```

```
# Define target and predictors
target = "Churn"
predictors = list(df.columns)
predictors.remove(target)

df[target].value_counts(normalize=True)
```

Churn	proportion
0	0.73463
1	0.26537

dtype: float64

Identify numeric and categorical predictors

```
numeric_cols = ["tenure", "MonthlyCharges"]
categorical_cols = [col for col in predictors if col not in numeric_cols]

print("Numeric columns:", numeric_cols)
print("Categorical columns:", categorical_cols)
```

```
Numeric columns: ['tenure', 'MonthlyCharges']
Categorical columns: ['OnlineSecurity', 'DeviceProtection', 'PaperlessBilling', 'Sen
```

Create X and y and perform train/test split

```
# Put the predictor and target values into arrays.
X = df[predictors]
y = df[target]
```

```
print('X shape: {}'.format(X.shape))
print('y shape: {}'.format(y.shape))
```

```
X shape: (7043, 7)
y shape: (7043,)
```

```
# Perform train/test split.
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42
)

print("Training size:", X_train.shape)
print("Test size:", X_test.shape)
```

```
Training size: (5282, 7)
Test size: (1761, 7)
```

We perform the train/test split before preprocessing to avoid data leakage.

Encoding and scaling are fit on the training data and then applied to the test data.

Create dummy variables for categorical predictors

```
# Dummy Variables
X_train = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
```

```
# Apply one-hot encoding to test data
X_test = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True)
```

```
# Align test columns to training columns
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
```

```
# Confirm Shapes
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X_train shape: (5282, 10)
X_test shape: (1761, 10)
```

Scale numeric features

```
scaler = StandardScaler()
```

```
# Scale numeric features
X_train[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test[numeric_cols] = scaler.transform(X_test[numeric_cols])
```

```
# Confirm scaling worked
```

```
X_train[numeric_cols].describe()
```

	tenure	MonthlyCharges
count	5.282000e+03	5.282000e+03
mean	-2.690431e-18	3.632081e-17
std	1.000095e+00	1.000095e+00
min	-1.323720e+00	-1.544085e+00
25%	-9.561813e-01	-9.730947e-01
50%	-1.394293e-01	1.871452e-01
75%	9.223484e-01	8.328295e-01
max	1.616588e+00	1.787247e+00

Baseline accuracy

Compute baseline from training data

```
baseline_accuracy = (y_train == 0).mean()
print("Baseline accuracy: {:.3f}".format(baseline_accuracy))
```

```
Baseline accuracy: 0.737
```

Model 1: Logistic Regression (default settings)

Import LogisticRegression

```
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train);
```

```
# Cross validation accuracy
cv_accuracy = cross_val_score(log_reg, X_train, y_train, cv=10).mean()
print("CV accuracy (default Logistic): {:.3f}".format(cv_accuracy))
```

```
CV accuracy (default Logistic): 0.791
```

Test accuracy (default Logistic Regression)

```
test_accuracy = log_reg.score(X_test, y_test)
print("Test accuracy (default Logistic): {:.3f}".format(test_accuracy))
```

```
Test accuracy (default Logistic): 0.802
```

Hyperparameter tuning for Logistic Regression

Define parameter grid

```
param_grid = {"C": [0.01, 0.1, 1, 10, 100]}
```

```
# Gird search
log_reg_cv = GridSearchCV(
    LogisticRegression(max_iter=1000),
    param_grid,
    cv=10
)

log_reg_cv.fit(X_train, y_train);

print("Best C:", log_reg_cv.best_params_)
print("Best CV accuracy: {:.3f}".format(log_reg_cv.best_score_))
```

```
Best C: {'C': 0.01}
Best CV accuracy: 0.792
```

Test accuracy (tuned Logistic Regression)

```
test_accuracy_tuned = log_reg_cv.score(X_test, y_test)
print("Test accuracy (tuned Logistic): {:.3f}".format(test_accuracy_tuned))
```

```
Test accuracy (tuned Logistic): 0.795
```

Model 2: KNN (default settings)

```
knn = KNeighborsClassifier()
knn.fit(X_train, y_train);
```

```
# Cross validation accuracy
cv_accuracy_knn = cross_val_score(knn, X_train, y_train, cv=10).mean()
print("CV accuracy (default KNN):{:.3f}".format(cv_accuracy_knn))
```

```
CV accuracy (default KNN):0.780
```

Test accuracy (default KNN)

```
test_accuracy_knn = knn.score(X_test, y_test)
print("Test accuracy (default KNN): {:.3f}".format(test_accuracy_knn))
```

```
Test accuracy (default KNN): 0.774
```

Determine best k using cross-validation

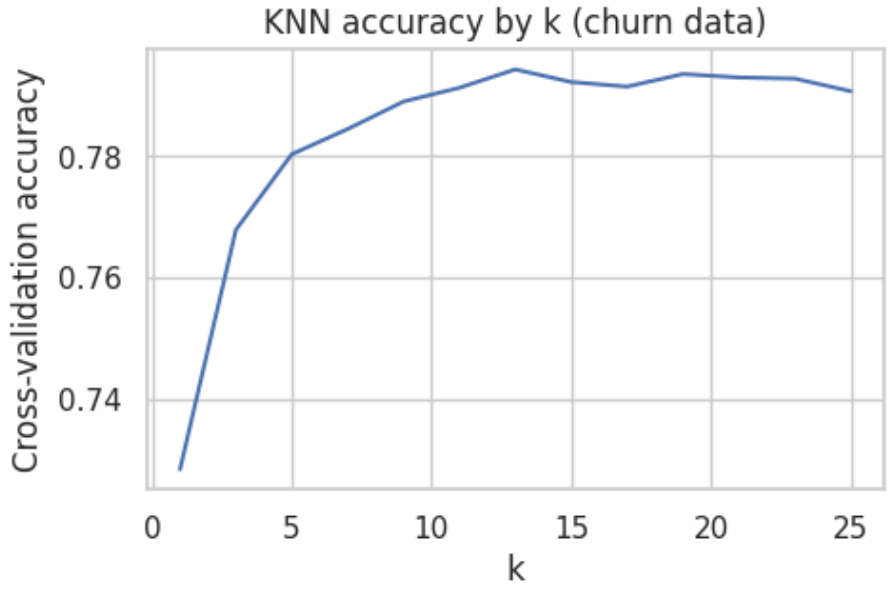
```
#Try odd k values
ks = list(range(1, 26, 2))   # 1,3,5,...,25
cv_scores = []

for k in ks:
    knn = KNeighborsClassifier(n_neighbors=k)
    score = cross_val_score(knn, X_train, y_train, cv=10).mean()
    cv_scores.append(score)
```

```
print(list(zip(ks, cv_scores)))
```

```
[(1, np.float64(0.7283188692215157)), (3, np.float64(0.7678860915392107)), (5, np.fl
```

```
#Plot
plt.plot(ks, cv_scores)
plt.xlabel("k")
plt.ylabel("Cross-validation accuracy")
plt.title("KNN accuracy by k (churn data)")
plt.show()
```



KNN with optimal k

```
#Train best KNN
best_k = ks[np.argmax(cv_scores)]
print("Best k:", best_k)

knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train);
```

```
Best k: 13
```

```
#Test accuracy with best k
test_accuracy_knn_best = knn_best.score(X_test, y_test)
print("Test accuracy (best KNN): {:.3f}".format(test_accuracy_knn_best))
```

```
Test accuracy (best KNN): 0.792
```

Conclusion

```
print("Baseline accuracy: {:.3f}".format(baseline_accuracy))
print("Logistic test accuracy (default): {:.3f}".format(test_accuracy))
print("Logistic test accuracy (tuned): {:.3f}".format(test_accuracy_tuned))
print("KNN test accuracy (default): {:.3f}".format(test_accuracy_knn))
print("KNN test accuracy (best k={}): {:.3f}".format(best_k, test_accuracy_knn_best))
```

```
Baseline accuracy: 0.737
Logistic test accuracy (default): 0.802
```

```
Logistic test accuracy (tuned): 0.795
KNN test accuracy (default): 0.774
KNN test accuracy (best k=13): 0.792
```

Both models improve on the baseline accuracy of 0.737. Logistic regression achieved the highest test accuracy (about 0.80), slightly outperforming KNN (about 0.79). Therefore, logistic regression is the preferred model for this problem.